Code Assessment
of the Core V2 Smart Contracts

PUBLIC

Date April 23, 2026

Produced for SYMBIOTIC

By CHAINSECURITY

Contents

1 Executive Summary

Dear all,

Thank you for trusting us to help Symbiotic with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of Core V2 according to **Scope** to support you in forming an opinion on their security risks.

Symbiotic implements the v2 of the Symbiotic restaking protocol. V2 redesigns the delegation and slashing architecture, replacing four delegator types with a single hierarchical slot tree, introducing an adapter system for external yield, bucket-based withdrawals, native ERC20 vault shares, and instant withdrawals.

The most critical subjects covered in our audit are slashing guarantees, functional correctness of the slash lifecycle, and asset solvency across the vault-delegator-slasher boundary. The general subjects covered are code complexity, migration correctness, and defensive programming practices.

The most notable findings, all of which have been addressed, include:

- **Slash lifecycle correctness:** `VaultV2.onSlash` **Reverts When Entire Slashed Amount Is Owed to Adapters** and **Claiming Withdrawals Exposes Networks to Adapters** led to slashing DoS. Both have been resolved.
- **Instant withdrawals** introduced multiple risks, including **Temporary Withdrawal DoS through Donation** and a cross-contract reentrancy enabling invariant violations (see **Delegator Hook Can Reenter Vault During Slash Execution**). The former has been partially corrected and the remaining risk has been accepted. The latter has been resolved.

Other lower severity findings have been resolved or their risk has been accepted. Most notable among these are **Networks Cannot Reset Allocation After Migration** and **Instant Withdrawal Fee May Be Unfair**.

In summary, we find that the codebase provides a satisfactory level of security. **System Considerations** contains various additional observations for users, curators, operators, networks, governance, integrators, and other interested parties.

The system is complex, with intricate state and deep interactions between the different elements. We recommend improving test coverage. Coverage is solid but does not exercise all edge cases of the system.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,
ChainSecurity

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the Core V2 repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

Version	Date	Commit Hash	Note
1	24 March 2026	90337df6119cb8e09d7f2eebb585d9028197c7db	Initial audit
2	8 April 2026	e2df19740a7347f00176d950dc9421b99b350eb5	After Intermediate Report
3	15 April 2026	32e7d270ff7796a14276d1c0fb3fdff3c6402572	Fixes
4	22 April 2026	50a329eb359e6bb06561a8807910fea4152dd556	Additional fixes

For the Solidity smart contracts, the compiler version `0.8.28` was chosen.

The following files are in scope of this review:

```
src/
└── contracts/
    ├── delegator/
    │   └── UniversalDelegator.sol
    ├── vault/
    │   ├── VaultV2.sol
    │   ├── VaultV2Migrate.sol
    │   ├── VaultV2Storage.sol
    │   └── common/
    │       └── ERC4626Math.sol
    ├── slasher/
    │   └── UniversalSlasher.sol
    ├── libraries/
    │   ├── CheckpointsV2.sol
    │   └── UniversalDelegatorIndex.sol
    └── AdapterRegistry.sol
```

2.1.1 Excluded from scope

Any file not explicitly listed in the Scope section is out of scope. In particular, external libraries (e.g. OpenZeppelin, Solady), deployment scripts, and tests are excluded.

Additionally, all configurations and operational usage are out of scope, including role assignments, the adapters, the adapter registry contents, the slot tree configurations, and the chosen epoch durations, veto durations, resolver set delays, deposit limits, or withdrawal buffer sizes. Further, it is expected that the contracts are properly configured and that all parties agree to that configuration.

3 System Overview

This system overview describes the initially received version (**Version 1**) of the contracts as defined in the Assessment Overview.

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

Symbiotic implements the second version of its core restaking contracts: a vault, a delegator, and a slasher, replacing the previous implementations with a unified architecture. Users deposit ERC20 collateral into a vault, the delegator allocates that collateral across networks and operators, and the slasher allows networks to slash misbehaving operators, sending slashed funds to a designated burner address.

3.1 Vault

`VaultV2` holds ERC20 collateral deposited by users and issues ERC20 shares in return. Users call `deposit()` to mint shares and `withdraw()` to request a redemption. Withdrawals use a bucket-based system: because the share-to-asset conversion rate can change within an epoch (due to slashes or donations), each price change opens a new bucket so that requests made at different rates are tracked separately. Claims become available after the epoch duration elapses via `claim()`. An instant withdrawal path (`instantWithdraw()`) is also available, limited by a configurable withdrawal buffer maintained by the delegator.

The vault supports an adapter system where idle collateral can be allocated to whitelisted external contracts (up to 10) through `allocateAdapters()` and `deallocateAdapters()`. Each adapter has its own allocation limit. Subvaults with the `noAdapters` flag reserve a portion of the vault's stake that adapters cannot touch. When collateral is locked in adapters during a slash and the vault cannot transfer the full amount, the slasher tracks the shortfall as an owed amount for later settlement via `syncOwedSlash()` on the slasher.

Deposits can be gated by a whitelist and capped by a configurable limit. The vault also supports fee collection through an external fee registry and a rewards contract.

Migration from v1 vaults converts epoch-indexed withdrawal mappings to buckets and deploys new delegator and slasher instances.

3.2 Delegator

`UniversalDelegator` replaces the four v1 delegator types with a single hierarchical slot tree. The tree has up to three levels: subvaults (depth 1, up to 10), networks (depth 2, up to 15 per subvault), and operators (depth 3, up to 20 per network). Each slot has a `size`, the maximum amount of collateral it can receive from its parent, set by the curator via `setSize()`.

A curator manages the tree through role-gated functions: `createSlot()` adds a new slot under a parent, `setSize()` adjusts a slot's size, `swapSlots()` reorders siblings, and `removeSlot()` removes a slot from the tree.

The root's balance is the vault's total stake (active stake plus active withdrawals). Collateral flows down the tree: each slot's effective allocation is the minimum of its `size` and what the parent can provide. In a non-shared subvault, siblings are filled left to right. The first child receives up to its `size` from the parent's balance, the second child receives up to its `size` from whatever remains, and so on. In a shared subvault, every child network can claim up to the full subvault balance, meaning the same collateral backs multiple networks simultaneously.

Size decreases are not immediate: the old size remains slashable for one epoch duration via a pending mechanism. The difference between the old and new size is tracked cumulatively and expires naturally after the epoch.

Subvaults carry two flags. `isShared` determines whether child networks share allocation or receive independent slices. `noAdapters` reserves the subvault's stake as liquid collateral that cannot be sent to adapters.

Networks opt in by calling `setMaxNetworkLimit()`. This can only be called once with `type(uint256).max` and cannot be changed afterward. Actual allocation sizing is controlled entirely by the curator. A network can opt out via `resetAllocation()`.

Stake queries (`stake()`, `stakeAt()`, `stakeFor()`) take a `duration` parameter that acts as a time window filter: it returns how much collateral is locked for at least the next `duration` seconds. A longer duration excludes withdrawals and pending decreases that will mature soon, giving a more conservative number. A duration of 0 includes everything currently slashable, while `epochDuration - 1` excludes almost all pending, returning only the long-term committed stake.

3.3 Slasher

`UniversalSlasher` implements a request-veto-execute slash lifecycle. A network's middleware calls `requestSlash()` specifying the subnetwork, operator and amount. A resolver may call `vetoSlash()` within a configured veto duration. After the veto window, anyone can call `executeSlash()` to finalize the slash. The veto duration must be shorter than the epoch duration. Changing a subnetwork's resolver is subject to a separate `resolverSetDelay` that must exceed the epoch duration.

`executeSlash()` caps the slash at the current slashable stake, then calls `onSlash()` on the delegator and on the vault in sequence. Since a slot's effective allocation is bounded by the parent's available balance, reducing the vault's total stake would naturally lower allocations across the tree but without targeting the correct slots, the reduction would land on whichever slots happen to be last in fill order rather than on the slashed operator. The delegator's `onSlash()` solves this by walking the operator, network, subvault path and reducing sizes and/or clearing pending at each level, ensuring the slash is attributed to the correct slots. In shared subvaults, a sibling's slash reduces the shared pool, which could lower a network's apparent allocation below what it was when a slash request was created. Guarantee accounting corrects for this in the slasher's view, so that each network can still be slashed up to its allocation at request time. The delegator returns the actual slashed amount, which may be less than requested. The vault's `onSlash()` then handles the collateral: it splits the slash proportionally between active stake and active withdrawals, reprices in-flight withdrawal buckets, and transfers the slashed collateral to the burner. If the subnetwork's subvault allows adapters, the vault attempts to deallocate first. Any shortfall that cannot be transferred becomes an owed amount.

The slasher tracks owed amounts per subnetwork and operator. Anyone can call `syncOwedSlash()` on the slasher to settle these when funds become available. The slasher also supports legacy slash requests created before migration.

3.4 Adapter Registry

`AdapterRegistry` is an owner-gated contract that maintains a whitelist of adapter contracts eligible to receive vault collateral.

4 Trust Model

The Core V2 contracts use role-based access control. Each vault, delegator, and slasher instance has its own set of role holders configured at initialization and by the Admin.

4.1 Global

- **Factory owner** — *Fully trusted.* Controls which implementations can be deployed and migrated to. Worst case: whitelist a malicious implementation that, once deployed or migrated to by a vault owner, drains all vault collateral.
- **Adapter registry owner** — *Fully trusted.* Controls which adapter contracts are eligible to receive vault collateral. Worst case: whitelist a malicious adapter that steals all allocated collateral.

4.2 Vault

- **Admin** (`DEFAULT_ADMIN_ROLE`) — *Fully trusted.* Can grant and revoke all other vault roles.
- **Curator** (`SET_ADAPTER_LIMIT_ROLE`, `SWAP_ADAPTERS_ROLE`, `ALLOCATE_ADAPTER_ROLE`, `DEALLOCATE_ADAPTER_ROLE`) — *Partially trusted.* Controls adapter allocation. Worst case: allocate all non-reserved collateral to adapters, making it temporarily unavailable for slashing.
- **Deposit roles** (`DEPOSIT_WHITELIST_SET_ROLE`, `DEPOSITOR_WHITELIST_ROLE`, `IS_DEPOSIT_LIMIT_SET_ROLE`, `DEPOSIT_LIMIT_SET_ROLE`) — *Partially trusted.* Controls who can deposit and up to what limit. Worst case: enable the whitelist and remove all depositors, preventing future deposits.

4.3 Delegator

- **Admin** (`DEFAULT_ADMIN_ROLE`) — *Fully trusted.* Can grant and revoke all other delegator roles.
- **Curator** (`CREATE_SLOT_ROLE`, `SET_SIZE_ROLE`, `SWAP_SLOTS_ROLE`, `REMOVE_SLOT_ROLE`, `SET_WITHDRAWAL_BUFFER_SIZE_ROLE`) — *Partially trusted.* Controls the slot tree structure and all allocation sizing. Worst case: resize or remove slots to redirect stake between networks. Cannot instantly free collateral from slashing due to the pending mechanism.
- **Hook setter** (`HOOK_SET_ROLE`) — *Partially trusted.* Can set the delegator hook contract called during slashes. Worst case: set a malicious hook that consumes the 250,000 gas budget on every slash.

4.4 Slasher

- **Resolver** — *Partially trusted.* Can veto pending slash requests within the veto duration. Worst case: veto all legitimate slashes, preventing a network from slashing misbehaving operators.

4.5 External Contracts

- **Adapter** — *Fully trusted.* Holds vault collateral. Worst case: does not return collateral.

4.6 Untrusted

- **Depositors / end users** — Can deposit, withdraw, claim, and instantly withdraw within the system's constraints.

5 System Considerations

We leverage this section to highlight findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require a modification inside the project. Instead, they should raise awareness in order to improve the overall understanding for users and developers.

SC1 Instant Withdrawal Considerations

System Consideration	Version 1

`instantWithdraw` is a novel feature that improves UX but introduces risks that integrators, curators, and users should be aware of. The following considerations apply.

Fees as Implicit Slash Insurance

Instant withdrawals pay a fee, which is distributed to the remaining users in the system. Economically, this fee can be viewed as a prepayment for potential future slashing: a curator setting the fee at 20% is signaling that they do not expect slashes in excess of 20%.

The impact on the remaining vault users depends on how the eventual slash compares to the fee that was paid at exit:

- **Slash < fee**: Remaining users are penalized *less* than they would have been if the exiting user had used a regular withdrawal.
- **Slash = fee**: Remaining users are penalized *equivalently*.
- **Slash > fee**: Remaining users are penalized *more* than they would have been if the exiting user had used a regular withdrawal.

`instantWithdraw` Disabled at 100% Fee

When `FEE_REGISTRY` returns a fee of `MAX_FEE` (100%), the entire withdrawn amount is consumed by the fee, so `fees` equals `withdrawnAssets`. `instantWithdraw` then calls `_safeTransferOut(recipient, 0)`, which reverts via `_revertIfZero`. As a result, `instantWithdraw` is effectively disabled whenever the fee is set to 100%.

The same condition can arise at non-maximum fees for sufficiently small `withdrawnAssets`, where rounding up on the fee calculation consumes the full amount and triggers the same revert.

SC3 Share Mechanism Considerations

System Consideration	Version 1

VaultV2 implements a share-based accounting mechanism. Consequently, considerations similar to ERC-4626 apply.

Integrators, users, curators, and all other related parties should be aware that integration is not trivial and the code must be well understood. The sections below outline common pitfalls.

Inflation and Deflation Possible

VaultV2 uses checkpoint-based accounting for vault and withdrawal shares rather than `balanceOf`, which eliminates the classic direct-transfer donation attack. However, share price inflation is still possible through two vectors:

1. `donate()`: Called by the `REWARDS` contract, increases `activeStake` without minting shares (e.g. through `IAdapterBase.skim()`).
2. `instantWithdraw()`: Fees are routed to `REWARDS` via `distributeDonationRewards`, which can cycle back as a `donate()` call, increasing share price for remaining holders.
3. Stealth-donation due to rounding errors might be possible.

The virtual offset adds `+1` to virtual assets and `+10**_decimalsOffset()` to virtual shares in the `fullMulDiv` conversion formulas. Newly deployed vaults use an offset of 6. While this increases the cost of precision-based attacks, it does not eliminate them.

Conversely, shares can also lose value: `onSlash()` reduces `activeStake` without burning shares, so each remaining share is worth proportionally less.

Withdrawal and Deposit DoS via Rounding

Share/asset conversions round up for withdrawals and round down for redemptions. For dust positions where the share price has appreciated, `previewWithdraw` can round up to more shares than the user holds, reverting with `TooMuchWithdraw`. Conversely, `previewRedeem` on a small number of shares can yield 0 assets, reverting via `_revertIfZero`. In both cases the position is economically negligible but permanently unrecoverable. Fully slashed vaults and similar edge cases where `totalAssets` drops sharply may exhibit the same behaviour.

This also applies to `claim`, which can revert due to `_safeTransferOut` reverting on 0-transfers.

Similarly, very small deposits may revert, as the minted share amount rounds to zero. The virtual share offset does not prevent this at low balances.

No Slippage Protection

Functions such as `deposit`, `withdraw`, and `redeem` are not slippage protected. The same applies to `claim`, though meaningful slippage protection there is inherently difficult because the redeemed amount is determined at withdrawal time. The virtual share offset provides some, but not complete, protection.

SC4 Migration to V2 Requires Careful Preparation by All Participants

System Consideration | Version 1

The V2 migration changes trust assumptions and operational models for networks, operators, and curators. These changes are not cosmetic. Participants who migrate without understanding the differences risk misconfigured slashing, loss of sizing control, or unintended adapter exposure.

Example of differences that require attention:

- **Slashing guarantees change.** V2 uses a rolling window. Networks must reassess what guarantee window they can rely on and adjust their middleware accordingly.
- **Withdrawal mechanics differ.** V2 replaces epoch-indexed withdrawals with bucket-based withdrawals. The timing of when withdrawals become claimable and how they interact with slashing has changed.
- **The delegator tree replaces flat allocation.** V1's four delegator types (NetworkRestake, FullRestake, OperatorSpecific, OperatorNetworkSpecific) are replaced by a single 3-level tree with shared/non-shared

modes. Curators must understand slot ordering, prefix sums, and the pending mechanism to configure allocations correctly.

- **Adapter risk is new.** V2 introduces adapters that hold vault collateral externally. Networks operating under `noAdapters = false` subvaults are exposed to adapter risk that did not exist in V1. The `noAdapters` flag must be set at subvault creation and cannot be changed later.
- **Networks lose sizing control.** In V1, `maxNetworkLimit` was an actual cap set by the network. In V2, it is forced to `type(uint256).max`, a pure boolean opt-in. The curator has unilateral control over how much stake a network receives. The network's only recourse is `resetAllocation`, which is all-or-nothing.
  - Note that the curator is responsible for creating an appropriate slot for the subnetwork within the subvault representing the migrated vault. The previous maximum network limit, as outlined above, only takes effect once the subnetwork slot has been created. If the curator attempts to assign the subnetwork to a different subvault, the network limit is set to 0, and the network must explicitly opt in again.

The migration itself is triggered by `VaultV2._migrate()`. It deploys a new `UniversalDelegator` and `UniversalSlasher`, converts withdrawal state, and cannot be reversed. However, it only creates a single initial subvault.

The curator must then manually rebuild the full slot tree (subvaults, networks, operators) via `createSlot` and `setSize` calls. Note that the delegator migration for `OperatorNetworkSpecificDelegator` will try to prepare all relevant slots but expects only one subnetwork to be registered (this assumption may not hold, but it covers many cases automatically).

The curator during and after the migration is highly trusted, and unexpected behaviour might occur regarding legacy slashes and configuration of the `UniversalDelegator`. Below is a non-exhaustive list of considerations:

- **Networks have weak guarantees for a limited time.** Legacy slash requests are routed through `UniversalDelegator.onSlashLegacy` to update the new state. However, if the configuration does not match the "legacy configuration", unexpected scenarios may occur. For example, parties unrelated to the slash may suddenly lose stake. Ultimately, guarantees regarding slashing can be violated.
- **Shared guarantees potentially violated.** Note that shared vault guarantees might be violated because the internal accounting may not track slashes correctly under certain configurations.
- **Overslashing possible.** Operators, curators, and vault users should be aware that they might be overslashed by a malicious network.
- **Last minute offences may be unslashable.** If an operator commits an offence just before migration and receives no delegation thereafter, they will be unslashable by the network.

To summarize, during a time window (some time before migration and one epoch duration after it), the trust guarantees cannot be provided. The curator and other parties are trusted to act responsibly.

SC5 Zombie Children After Slot Removal in UniversalDelegator

System Consideration | Version 1

`_removeSlot` sets `slot.exists = false` on the removed slot and unlinks it from its parent's linked list, but does not clear or mark its children. The children retain `exists = true` and their `size`, linked-list pointers, and other state remain intact.

As a result, operations that check `_revertIfNotExists` (such as `setSize`, `swapSlots`, `createSlot`, and `removeSlot`) still accept these orphaned children as valid targets. `getSlot` also returns them as if they are active.

This has no functional impact because the removed parent is severed from the tree. `getAllocated` for any zombie child returns 0 since the parent's balance in the tree is 0. The children are operationally inert despite appearing to exist.

SC6 Adapter Trust Assumptions

System Consideration | Version 1

Adapters are external contracts that hold vault collateral to generate yield. They are whitelisted by the `AdapterRegistry` owner and granted `ALLOCATE_ADAPTER_ROLE` and `DEALLOCATE_ADAPTER_ROLE` on the vault. The following assumptions about adapter behavior are the most important necessary for the vault to function correctly. An adapter that violates any of these assumptions can disrupt deposits, withdrawals, slashing, or accounting.

- **`allocate`, `deallocate`, and `skim` must not revert.** `deposit` auto-allocates to `adapters[0]`, and `claim`, `instantWithdraw`, `onSlash`, and `deallocateAdapters` all call `deallocate` or `skim`. A reverting adapter blocks the calling function entirely. The `IAdapterBase` interface documents this: all three are annotated with "Must not revert."

- **Adapters must return funds on `deallocate` and must not incur permanent losses.** The vault tracks `adaptersAllocated` as the nominal amount held by adapters. When `_deallocateAdapter` is called, it calls `deallocate(amount)`. If the adapter returns zero (e.g., lock-up period, withdrawal queue, bad debt), the call succeeds but no collateral is recovered and `adaptersAllocated` is not reduced. If the adapter returns a nonzero value but does not actually transfer tokens, `_safeTransferIn` reverts. In either case, `adaptersAllocated` remains above the recoverable amount, `_adaptersOwe()` can never reach zero, and operations that require `adaptersAllocated <= _maxAllocatable()` such as `instantWithdraw` are blocked. `deallocateAdapters` is called in `claim`, `instantWithdraw`, and `onSlash` (when `withAdapters` is true). A broken or malicious adapter can therefore block user withdrawals and slash settlement.

- **Collateral token must not enable fee-on-transfer.** `_allocateAdapter` checks that the adapter received the full amount and reverts with `FeeOnTransferNotSupported` otherwise. If a token enables fees after vault deployment, allocation reverts. Deallocation also becomes problematic: `_safeTransferIn` pulls `deallocated` tokens from the adapter via `safeTransferFrom`, but the vault receives `deallocated - fee` due to the transfer tax. `adapterAllocated[adapter]` and `adaptersAllocated` both decrease by the full `deallocated` amount, so the vault's actual token balance ends up lower than what its accounting expects.

- **Adapters should optimally skim full balance**. If skimming does not fully skim what could be skimmed, reward stealing might be possible. Note that this also applies to deallocations. However, the involved parties will be incentivized to skim frequently to ensure that the skimmed amounts contribute to stake as soon as possible.

- **Adapters must not deploy collateral into systems secured by the vault's own stake.**

SC7 Slashing Cost May Exceed Slashed Value for Small Amounts

System Consideration | Version 1

`executeSlash` involves multiple external calls: `slashableStake` reads from the delegator, `delegator.onSlash` traverses the full slot tree updating checkpoints at each level, `VaultV2.onSlash` updates `activeStake`/withdrawals and potentially calls `deallocateAdapters`, and `_burnerOnSlash` may invoke a burner hook. For small slash amounts, the gas cost of this call chain can exceed the value of the slashed tokens by much. Networks should be aware that onboarding operators with very little stake can make slashing economically irrational.

SC8 Slashing Guarantees

System Consideration | Version 1

A core invariant of any restaking protocol is the **slashing guarantee**: when a network captures an operator's stake at time T, that stake must remain slashable for a known window W. This lets the network rely on the captured amount as economic security. If the actual guarantee is weaker than what networks expect, a malicious operator can misbehave at the edge of the real window and escape slashing. A rogue curator can reduce allocations and free collateral before the network reacts. Depositors can withdraw, shrinking the vault's backing below what networks rely on. The protocol's security promise to networks no longer matches reality.

The subsections below outline the mechanism and highlight important considerations for networks, stakers, curators, and operators.

Networks Pick Their Guarantees

`UniversalDelegator` provides `stakeFor` for querying the stake that will still be slashable after `duration` seconds.

Naively choosing `duration = 0` when computing operator power leads to practically no guarantees being available. Consider the following example:

1. A user creates a withdrawal at $t - E + 1$, where E is the epoch duration. At t, it is not yet claimable.
2. The operator, still holding that power in the network, commits a slashable offence at t.
3. At $t + 1$, nothing is left to slash since the withdrawal is now claimable.

A network defining a slashing window W means that the stake which has not become claimable by $t + W$ should be considered as the voting power, so that the voting power can be slashed within W. Equivalently, only withdrawals (and operator downsizing) created no earlier than $t + W - E - 1$ can be counted.

Thus, the capture of the operator's stake at time T depends heavily on the choice of W. That is, the operator's power should be queried via `stakeFor` with a `duration` of at least W.

Networks must therefore carefully choose their slashing window, which also has implications for the guarantees they receive.

No Capture Time Leaves a Vulnerable Window

The lack of capture timestamps can shrink the effective slashing window, because the publication of a slash request is itself delayed by reaction time and the veto period. Consider the following example:

1. At $t - x$, a slashable offence is committed. The operator's power was queried via `stakeFor` with `duration = W`. The full slash can therefore only occur up to $t - x + W$.
2. The network requests a slash at t.
3. The slash can be executed at the earliest at $t - x + W + 1$, due to the veto period.
4. By then, the captured stake is already claimable and therefore unslashable.

As a consequence, networks should account for reaction time, veto period, and similar latencies when defining the duration they pass to `stakeFor`. The slashing window will typically be smaller than that duration.

To summarize, **the duration passed to `stakeFor` must be larger than the desired slashing window**, so it absorbs reaction time and the veto period.

Stakers Are Forgiving and Ready for Sacrifices

Stakers are gracious toward networks.

More specifically, the `UniversalSlasher` allows the following:

- Slashes within a window of one epoch duration E.
- It offers all the stake currently assigned to the operator for the subnetwork at request time, even if this does not necessarily correspond to the captured stake at offence or request time. As long as W is meaningful and respected by the network, the captured stake can still be slashed.
- If any of the previously outlined scenarios occur, stakers absorb the difference and allow slashing up to the amount that is available. If the stake reduction did not affect the operator, the slash is fully executable; otherwise, it is granted partially.
- For withdrawals, stakers accept that a delayed network may slash against a capture where part of the stake responsible for the slash is no longer available. The remaining stakers and new depositors then cover the cost where possible.

To summarize, **if the network is delayed, as much as possible will be slashed and the remaining stakers will cover the cost as much as possible**. Operators will, of course, be penalized accordingly.

SC9 Withdrawal Claimability and Adapter Debt

System Consideration | Version 2

When collateral is allocated to adapters, claiming a withdrawal requires leaving enough liquid funds for both the no-adapters reserve and any outstanding owed slashes. Otherwise, the claim reverts.

Withdrawers should be aware that:

- The waiting time may be prolonged if funds cannot be deallocated.
- An outstanding owed slash further delays claims until it is settled.
- If little liquidity is available, a race condition for withdrawers is created.

Additionally, due to rounding errors, `_unclaimedRaw` accumulates a small buffer of a few wei, which contributes marginally to easier withdrawals.

Additionally, note that the no-adapters reserve will typically be higher than expected. This is mainly due to `getNoAdaptersSize` being an overapproximation of the allocations made to no-adapter subvaults.

6 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- **Likelihood** represents the likelihood of a finding to be triggered or exploited in practice
- **Impact** specifies the technical and business-related consequences of a finding
- **Severity** is derived based on the likelihood and the impact

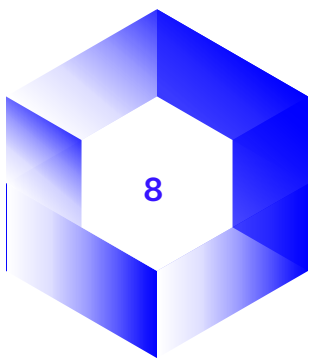
We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	**Impact**		
	High	Medium	Low
High	**Critical**	**High**	**Medium**
Medium	**High**	**Medium**	**Low**
Low	**Medium**	**Low**	**Low**

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

7 Findings

In this section, we describe the findings identified during the course of the engagement. The findings are categorized by severity as explained in the Terminology section. Below we provide an overview of the total number of findings per severity, as well as the number of open issues.



Open Issues

Total number of findings

Critical | High | Medium | Low | Informational

7.1 Overview

The following table lists all identified findings along with their severity and current status. A finding is considered resolved once it has been fully corrected or the specification has been changed accordingly. Findings with any other status, including partial fixes, acknowledgements, and accepted risks, remain open.

High	#001	`VaultV2.onSlash` Reverts When Entire Slashed Amount Is Owed to Adapters	Code Corrected
High	#015	Claiming Withdrawals Exposes Networks to Adapters	Code Corrected
Medium	#014	Delegator Hook Can Reenter Vault During Slash Execution	Code Corrected
Medium	#016	Temporary Withdrawal DoS Through Donation	Partially Corrected / Risk Accepted
Low	#003	`resetAllocation` Makes Pending Slash Requests Permanently Unexecutable	Risk Accepted
Low	#004	Delegator Hook Receives Requested Slash Amount Instead of Actual Slashed Amount	Code Corrected
Low	#005	Networks Cannot Reset Allocation After Migration Until Curator Creates Their Slot	Risk Accepted
Low	#006	Shared Subvault Overguarantee	Code Corrected
Low	#007	Withdrawal Buffer Size Is Not a Strict Constraint on Instant Withdrawals	Risk Accepted
Low	#017	Instant Withdrawal Fee May Be Unfair	Risk Accepted
Info	#008	`_deallocateAdapter` Uses Adapter-Reported Amount Instead of Actual Transfer Delta	Code Corrected
Info	#009	`VaultV2.burner` Can Be `address(0)`	Risk Accepted
Info	#010	Dead Code	Code Corrected
Info	#011	Missing `nonReentrant` Modifiers	Risk Accepted
Info	#012	Withdrawal Buffer `prevSlot` Only Maintained by `swapSlots`	Acknowledged
Info	#013	Entire Functions Wrapped in `unchecked` Blocks	Code Corrected
Info	#018	`onSlash` Over-Reduces Parent Slots When `amount > allocated`	Code Corrected
Info	#019	`skimAdapters` Not Called in `instantWithdraw`	Code Corrected

Info	#020	Missing Role Grants in Initialization	Code Corrected

7.2 Descriptions

#001 `VaultV2.onSlash` Reverts When Entire Slashed Amount Is Owed to Adapters

Correctness	High	Version 1	Code Corrected

When a slash is executed on a vault with adapters (`withAdapters == true`), `VaultV2.onSlash` splits `slashedAmount` into two parts: what can be transferred to the burner immediately, and what remains owed because collateral is locked in adapters. The owed portion is determined by `_adaptersOwe()`, which returns how much `adaptersAllocated` exceeds `_maxAllocatable()`. This is collateral that should no longer be in adapters but cannot be pulled back instantly.

```
if (withAdapters) {
    deallocateAdapters();
    owedAmount = Math.min(slashedAmount, _adaptersOwe());
}

_safeTransferOut(burner, slashedAmount - owedAmount);
```

The immediate portion, `slashedAmount - owedAmount`, is sent to the burner via `_safeTransferOut`. However, `_safeTransferOut` reverts on a zero amount:

```
function _safeTransferOut(address to, uint256 amount) internal {
    _revertIfZero(amount);
    collateral.safeTransfer(to, amount);
}
```

If `_adaptersOwe() >= slashedAmount`, the entire slash is owed and nothing needs to be transferred immediately. `slashedAmount - owedAmount` equals zero, and the call reverts with `InsufficientAmount` instead of completing with the full amount owed.

The `UniversalSlasher` relies on the `owedAmount` returned by `onSlash` to update its `totalOwed` and per-subnetwork/operator `owed` accounting. Because `onSlash` reverts, these owed amounts are never recorded, and the slash cannot be settled later via `syncOwedSlash`.

Code Corrected

The code has been corrected. Fund transfers are only attempted if the amount to transfer to the burner is non-zero.

#015 Claiming Withdrawals Exposes Networks to Adapters

Security	High	Version 1	Code Corrected

Curators can set up subvaults with the `noAdapters` flag. Subnetworks that opt into such vaults (i.e. with their limit set) expect not to be exposed to adapter risk: slashes execute right away and funds are sent to the burner. However, that expectation may be violated.

Consider the following initial setup:

- Vaults V1 and V2 with respective subnetworks N1 and N2 where V1 is a no-adapter vault.
- Total stake is 100 and all have a size of 50 so that the allocation to N1 and N2 is 50.
- 50 is allocated to adapters since that is allowed.

Now consider the next:

- A user withdraws 50.
- Eventually the withdrawal is claimable. That increases `_adaptersOwe` to 50 since `_maxAllocatable` drops to 0. The resulting allocation is that only N1 retains 50 stake allocated to it.
- The user now claims their withdrawal, leaving 0 tokens in VaultV2.

The invariant that the VaultV2 token balance is not dropping below the no-adapter allocations is violated. Technically, a broader invariant is implemented, restricting that the balance falls below the no-adapter size.

As a consequence, no-adapter networks are exposed to adapter risks. They will not be able to slash if funds cannot be deallocated (e.g. timelocks, loss of funds).

To summarize, a crucial adapter-related invariant is violated.

Code Corrected

The code has been corrected. More specifically, the following is defined now in `claim`:

```
if (uint256(int256(withdrawals(withdrawalBucket()) - activeWithdrawals()) +
    _unclaimedRaw) < _adaptersOwe()) {
    revert InsufficientAmount();
}
```

That utilizes the previously unused `_unclaimedRaw`.

The meaning is the following: the sum of all unclaimed claimable withdrawals must be more than or equal to what the adapters owe. Mainly, that means that withdrawers have to await adapter deallocations accordingly.

#014 Delegator Hook Can Reenter Vault During Slash Execution

Security	Medium	Version 1	Code Corrected

In `UniversalSlasher.executeSlash`, the delegator's `onSlash` is called before the vault's `onSlash`. Inside `UniversalDelegator.onSlash`, after updating slot allocations, a custom hook is called:

```
address hook_ = hook;
if (hook_ != address(0)) {
    bytes memory hookCalldata = abi.encodeCall(IDelegatorHook.onSlash, (subnetwork,
        operator, amount, data));
    // ...
    assembly ("memory-safe") {
        pop(call(HOOK_GAS_LIMIT, hook_, 0, add(hookCalldata, 0x20), mload(hookCalldata), 0,
            0))
    }
}
```

Although both `UniversalDelegator.onSlash` and `VaultV2.instantWithdraw` are `nonReentrant`, they use separate reentrancy locks on different contracts.

By the time the hook fires, `UniversalDelegator.onSlash` has already reduced the slashed slot's size. Since the withdrawal buffer's available amount is derived from the vault's total balance minus what is allocated to slots, reducing a slot's size increases the withdrawal buffer by the same amount. The vault's `activeStake` has not yet been reduced (that happens later in `VaultV2.onSlash`), so the full pre-slash stake becomes instant withdrawable.

A malicious hook could therefore call `vault.instantWithdraw` in the hook to extract the newly freed withdrawal buffer.

Currently, this is mitigated by the `HOOK_GAS_LIMIT` of 250,000 gas, which might be insufficient to execute an `instantWithdraw` (involving ERC-20 transfers, delegator calls, and reward distribution). However, this is a gas-based mitigation rather than an invariant-based one. If `HOOK_GAS_LIMIT` is increased in a future version, this becomes exploitable.

> **Code Corrected**
>
> The hook was removed.

#016 Temporary Withdrawal DoS Through Donation

Design	Medium	Version 2	Partially Corrected / Risk Accepted

As outlined in Share Mechanism Considerations, the share price of withdrawal shares and vault shares can be inflated.

For example, assume the following scenario:

1. No users exist. Alice deposits 1 wei of assets and receives 1 wei of vault shares.
2. Alice withdraws directly after that. Alice now holds 1 wei of withdrawal shares corresponding to 1 wei of assets.
3. Alice deposits 10M tokens, assuming 18 decimals (`1e25`).
4. Alice calls `instantWithdraw` for `1e25`. Assume the instant withdrawal fee is 10%.
5. The withdrawal bucket is still at 1 share, but it now corresponds to `1e24+1` assets.

Over time, other depositors join and eventually attempt to withdraw. Their calls route through `ERC4626Math.previewDeposit`, which computes `assets.fullMulDiv(2, 1e24+2)`. As long as the withdrawn amount is less than roughly 500k tokens, the preview returns 0, and `_withdraw` then reverts via `_revertIfZero`.

Ultimately, a well-capitalized attacker can DoS withdrawals and redemptions.

Stuck users have two options:

- Use `instantWithdraw` to skip the withdrawal flow, at the cost of paying the fee.
- Wait until Alice's withdrawal becomes claimable, then wait for (or manually trigger) a slash, instant withdrawal, or adapter skim that opens a new bucket.

In any case, UX is severely degraded, and unknowing users may assume their funds are stuck.

> **Partially Corrected**
>
> The virtual shares mechanism has been scaled up. More specifically, a `_decimalsOffset` as commonly used in modern ERC-4626 vault implementations has been introduced:

```
function _decimalsOffset() internal view virtual override returns (uint8) {
    return migrateTimestamp == 0 ? super._decimalsOffset() : 0; // super._decimalsOffset()
        == 6
}
```

> However, for migrated vaults, the offset remains 0, allowing for easier DoS attacks on withdrawals. For new vaults, the DoS is now more difficult. For example, in the above example, it would be sufficient to withdraw roughly `1e18` assets to bypass the DoS.
>
> Of course, an attacker with sufficient capital could nonetheless create a DoS scenario. However, the attack will incur financial losses in both cases (migrated and non-migrated vaults). For migrated vaults, they will be significantly higher, leading to less economic incentive. However, if withdrawal DoS is in some form profitable, the profits may nonetheless exceed the cost.
>
> Symbiotic is aware of the remaining risk and accepts it.

#003 `resetAllocation` Makes Pending Slash Requests Permanently Unexecutable

Security	Low	Version 1	Risk Accepted

When a network (or its middleware) calls `resetAllocation`, `_removeSlot` pushes `_networkToSlot[subnetwork]` to 0. Any slash request that was already queued for that subnetwork becomes permanently stuck: every future `executeSlash` call reverts instead of completing.

`executeSlash` calls `delegator.onSlash`, which calls `getSlotOf` with the returned index 0 and then `getParentIndex(0)`, which reverts with `ZeroIndex`. Pre-migration, `getIsNoAdapters` calls `getSlotOfNetwork`, which returns 0 and reverts with `NotAssigned`.

An operator who observes or predicts a `resetAllocation`, for example by watching the mempool or knowing the network has announced its departure, can misbehave during the window before the reset lands (equivocate, censor, go offline) without economic consequence:

- If `resetAllocation` lands before `requestSlash`: `slashableStake` returns 0, and `requestSlash` reverts with `InsufficientSlash`.
- If `resetAllocation` lands during the veto window: `executeSlash` reverts permanently as described above.

In both cases the operator escapes slashing entirely. No coordination with the network is required; the operator simply exploits the predictable reset to act without consequence.

> **Risk Accepted**
>
> Symbiotic is aware of the risk and has documented the behavior. Networks should cautiously reset allocations

#004 Delegator Hook Receives Requested Slash Amount Instead of Actual Slashed Amount

Correctness	Low	Version 1	Code Corrected

`UniversalDelegator.onSlash` computes `actualAmount` as it walks the slot tree. At each level, `actualAmount` is capped by the slot's available pending and size. The function returns `actualAmount` to the caller.

The hook, however, receives `amount`:

```
bytes memory hookCalldata = abi.encodeCall(IDelegatorHook.onSlash, (subnetwork, operator,
    amount, data));
```

> **Code Corrected**
>
> The hook was removed.

#005 Networks Cannot Reset Allocation After Migration Until Curator Creates Their Slot

Design	Low	Version 1	Risk Accepted

`resetAllocation` allows a network (or its middleware) to remove its own slot from the delegator, opting out of the vault. It looks up the network's slot via `getSlotOfNetwork`, which reads from `_networkToSlot`:

```
function resetAllocation(bytes32 subnetwork) public {
    ...
    uint96 index = getSlotOfNetwork(subnetwork);
    if (index == 0) {
        revert NotAssigned();
    }
    ...
}
```

When a vault migrates from V1 to V2, `migrate` creates a single subvault but does not populate `_networkToSlot` for the networks that were active in the old delegator. The `_networkToSlot` entry for a network is only written when the curator calls `createSlot`.

Until the curator creates a slot for a given network, `getSlotOfNetwork` returns 0 and `resetAllocation` reverts with `NotAssigned`. A network that was actively opted in via `maxNetworkLimit` in the V1 delegator cannot call `resetAllocation` to opt out after migration until the curator explicitly creates its slot in the new delegator.

During this gap the network has no allocation and cannot be slashed, since `stakeFor` and `getAllocated` both return 0 without a slot. The network only becomes slashable again once the curator creates its slot, at which point `resetAllocation` also becomes available.

> **Risk Accepted**
>
> Symbiotic is aware and accepts the risk.

#006 Shared Subvault Overguarantee

Correctness	Low	Version 1	Code Corrected

In shared subvaults, when a network is slashed, the subvault's shared pool (pending and size) decreases. Other networks receive "guarantees" to compensate for the reduced pool. The slashed network must consume its own guarantees so it does not also receive credit for its own slash. This consumption rule allows networks to slash as if no other slash had occurred within the given timeframe.

However, the guarantees may be unnecessarily high.

Consider the following example:

- Assume a vault with stake 100 and one subvault with size 200. Assume two subnetworks N1 and N2 with each having an operator O1 and O2, respectively.
- O1 gets slashed for 100. The global slashing cursor and N1's cursor progress accordingly. Thus, N2 receives a guarantee of 100. `slashableStake(O1) == 0` and `slashableStake(O2) == 100` holds thereafter.
- O2 gets slashed for 100 (possible due to guarantee). The global slashing cursor and N2's cursor progress accordingly. The delta between the network cursors and the global cursor for both networks is the same and corresponds to 100 (in the context of N1, N1 received more guarantees).
- Since the sizes still allow for up to 100 allocation to each, the following hold: `slashableStake(O1) == slashableStake(O2) == 100`.
- They can get slashed again.

Note that this example can be constructed in various ways. For example, an operator without any stake may have slashable stake due to such overguarantees, allowing networks to slash them and reduce their size.

The root cause is that the global cursor does not progress in step with the actually slashed amounts. As a consequence, as soon as the slashed total exceeds the subvault's allocation, overguaranteeing can occur when the sizes permit it.

> **Code Corrected**
>
> The code has been corrected. At the shared subvault level, the amount used to reduce the size and pending is now capped at the subvault's allocation before being applied.

#007 Withdrawal Buffer Size Is Not a Strict Constraint on Instant Withdrawals

Design	Low	Version 1	Risk Accepted

The withdrawal buffer is a remainder of stake that is capped by the buffer's configured size. However, the size is not a strict cap. More specifically, if sufficient stake is available, the buffer is automatically refilled. Similarly, the buffer could be refilled by the fees paid. Since no size adjustment is performed, the buffer may grow again right after consuming the full buffer.

> **Risk Accepted**
>
> Symbiotic is aware and accepts the risk.

#017 Instant Withdrawal Fee May Be Unfair

Correctness	Low	Version 2	Risk Accepted

As outlined in Instant Withdrawal Considerations, the instant withdrawal fee serves as implicit insurance against potential slashes. However, because the fee is shared with pending withdrawals, including those that are potentially no longer subject to slashing, the distribution can shift the burden of a slash onto the remaining active stakers. Consider the following example:

1. A total stake of 300 exists: Alice is eligible for 200, and Bob is eligible for 100.

2. Alice requests a regular withdrawal of 100. Her active stake drops to 100, and a withdrawal of 100 is created.
3. Nearly one full epoch passes. Alice's pending withdrawal is no longer contributing to operator power.
4. A slash is requested for 10% of the 200 currently contributing to operator power (i.e. a fixed amount of 20).
5. Before the slash executes, Alice performs an instant withdrawal of her remaining 100 active stake and pays a 10% fee (10). The fee is split evenly: 5 is credited to active stake (Bob) and 5 is credited to Alice's pending withdrawal. Alice receives 90 from the instant withdrawal.
6. Alice's pending withdrawal becomes claimable and she claims 105. In total, Alice has received 195.
7. The slash of 20 executes on the remaining active stake. Bob's balance drops from 105 to 85.

Without the instant withdrawal, the slash of 20 would have been split pro-rata across Alice's 100 and Bob's 100 in active stake, leaving Bob with 90. Because part of the fee is credited to a withdrawal becoming claimable soon, the protection the fee was meant to provide to active stakers is diluted, and Bob absorbs more of the slash than he otherwise would.

Routing the fee *exclusively* to active stake would create the opposite problem: pending withdrawals that *are* still within the slashing window would be penalized more than their fair share.

In summary, the instant withdrawal fee mechanism can still produce unfair outcomes for remaining stakers, depending on epoch timing and the distribution of pending withdrawals at the moment of exit.

> **Risk Accepted**
> Symbiotic is aware and accepts the risk.

#008 `_deallocateAdapter` Uses Adapter-Reported Amount Instead of Actual Transfer Delta

Informational	Version 1	Code Corrected

`_allocateAdapter` checks the balance delta after transferring to an adapter and reverts with `FeeOnTransferNotSupported` if the received amount differs. `_deallocateAdapter` has no equivalent check: it calls `_safeTransferIn` but discards the return value, subtracting the adapter's self-reported `deallocated` from `adapterAllocated` and `adaptersAllocated`.

Some ERC-20 tokens support toggling transfer fees after deployment (e.g., USDT has a fee mechanism that is currently set to zero but can be activated). If the collateral token activates fees after the vault has already allocated to adapters, `_allocateAdapter` would start reverting (protecting future allocations), but `_deallocateAdapter` would silently over-count returned collateral. The vault's accounting would record more available balance than it actually holds, breaking the invariant `balance + adaptersAllocated >= activeStake + activeWithdrawals`.

> **Code Corrected**
> Symbiotic reduces the allocation now. Hence, such hypothetical losses are treated equivalently as strategies with losses. The consequence is clear: the adapter might be unremovable in such scenarios. However, through small donations this can be resolved.

#009 `VaultV2.burner` Can Be `address(0)`

Informational	Version 1	Risk Accepted

`burner` is set during initialization and is not validated against `address(0)`. When a slash is executed, `_safeTransferOut(burner, ...)` calls `collateral.safeTransfer(address(0), ...)`. The result depends on the collateral token implementation: OpenZeppelin's ERC20 reverts on transfers to `address(0)`, blocking all slashing. Solady's ERC20 allows it, so the slash succeeds.

> **Risk Accepted**
> Symbiotic is aware and accepts the risk.

#010 Dead Code

Informational	Version 1	Code Corrected

The following code is defined but never read or used:

- The following error selectors are defined in `IVaultV2` but never used: `AlreadySet`, `InsufficientClaim`, `InsufficientRedemption`, `InsufficientWithdrawal`, `InvalidCaptureEpoch`, `InvalidClaimer`, `InvalidLengthEpochs`, `InvalidOnBehalfOf`, `InvalidRecipient`, `MissingRoles`.
- `_unclaimedRaw` is an `internal` variable. It is incremented when claimable withdrawals are frozen into a bucket (`_updateWithdrawalsSharePrice` line 529) and decremented when users claim (`claim` line 390). No view function, allocation check, or business logic reads this variable.
- `IUniversalSlasher` declares four errors that are never reverted in the implementation: `AlreadySet`, `InvalidCaptureTimestamp`, `NoResolver`, `SlashRequestNotExist`. For `SlashRequestNotExist` specifically, out-of-bounds array access produces a Solidity panic (0x32 index OOB) instead of the custom error. External integrators reading the interface may expect and handle these custom errors but will never encounter them.

> **Code Corrected**
> All of the above have been removed. `_unclaimedRaw` now has a usecase.

#011 Missing `nonReentrant` Modifiers

Informational	Version 1	Risk Accepted

In `UniversalDelegator`, `resetAllocation` does not have a `nonReentrant` modifier. Similarly, other functions are not reentrancy protected.

In `VaultV2`, `deallocateAdapters` and `skimAdapters` are permissionless and callable while the vault is in the middle of a `nonReentrant`-protected operation such as `deposit`. When the vault's collateral is a token with transfer callbacks (e.g. ERC777), a depositor could reenter during `safeTransferFrom` and call `skimAdapters` or `deallocateAdapters` before the deposit state is finalized.

> **Risk Accepted**
> Symbiotic is aware and accepts the risk.

#012 Withdrawal Buffer `prevSlot` Only Maintained by `swapSlots`

Informational | **Version 1** | **Acknowledged**

The withdrawal buffer (`WITHDRAWAL_BUFFER_CHILD_INDEX = 0xFFFFFFFF`) sits at the tail of the depth-1 linked list. `createSlot` and `_removeSlot` never update its `prevSlot` .

`swapSlots` is the only code path that writes to `WB.prevSlot` , when `slot2` is `lastChild` , the swap moves `slot1` to the last position and sets `slots[WB].prevSlot = index1.getChildIndex()` . The write is logically correct for the swap but inconsistent with the rest of the codebase. A subsequent `createSlot` appending a new slot after the swapped one will not update `WB.prevSlot` , leaving it stale again.

However, note that there is no functional impact.

> **Acknowledged**
> Symbiotic is aware and acknowledged the finding.

#013 Entire Functions Wrapped in `unchecked` Blocks

Informational | **Version 1** | **Code Corrected**

Across `VaultV2` , `UniversalDelegator` , and `UniversalSlasher` , nearly every function body is wrapped in a single `unchecked` block. This disables Solidity's built-in overflow and underflow checks for all arithmetic in those functions, including lines where overflow is not obviously impossible.

For example, in `VaultV2.deposit` :

```
function deposit(address onBehalfOf, uint256 amount) public nonReentrant returns (...) {
    unchecked {
        // ...
        uint256 newActiveShares = curActiveShares + mintedShares;
        require(newActiveShares >= curActiveShares);   // ← manual overflow check
        _activeShares.push(uint48(block.timestamp), newActiveShares);
        _activeStake.push(uint48(block.timestamp), curActiveStake + depositedAmount);  // ←
            no check
        _activeSharesOf[onBehalfOf].push(uint48(block.timestamp),
            activeSharesOf(onBehalfOf) + mintedShares);  // ← no check
    }
}
```

The same pattern repeats across the codebase. Functions like `_withdraw` , `_updateWithdrawalsSharePrice` , `onSlash` , `_initialize` , `_migrate` in `VaultV2` , and `createSlot` , `setSize` , `onSlash` , `swapSlots` in `UniversalDelegator` are all fully unchecked. Each contains a mix of arithmetic that genuinely cannot overflow and arithmetic that relies on implicit assumptions to not overflow. Distinguishing the two requires careful line-by-line analysis, and any future modification to these functions risks introducing silent overflow bugs.

A non-exhaustive list of arithmetic operations where overflow or underflow might be possible:

- `VaultV2.onSlash` : `curActiveWithdrawals - (slashedAmount - activeSlashed)` .

- `VaultV2._updateWithdrawalsSharePrice`: `withdrawals(bucket) - activeWithdrawals()` and `withdrawalShares(bucket) - curActiveWithdrawalShares`.
- `VaultV2._migrate`: `collateral.balanceOf(address(this)) - activeStake() - curActiveWithdrawals`.
- `UniversalSlasher.executeSlash`: `slashedAmount - owedAmount`.
- `UniversalSlasher.syncOwedSlash`: `curOwed - slashedAmount` and `totalOwed -= slashedAmount`.
- `UniversalDelegator._getPendingSize`: `size + pending`
- `UniversalDelegator._getPrevSum`: `_getPrevSizeSum + _getPrevPendingSum`

We recommend restricting `unchecked` to the specific arithmetic operations where overflow is proven impossible, rather than wrapping entire function bodies.

> **Code Corrected**
> The relevant `unchecked` blocks have been removed.

#018 `onSlash` Over-Reduces Parent Slots When `amount > allocated`

Informational	Version 1	Code Corrected

In practice the slasher caps `amount` to `slashableStake` before calling `onSlash`, so this condition should not arise under normal operation. However, `onSlash` itself has no defensive check against it, and future refactors to the slasher logic could expose this path.

`UniversalDelegator.onSlash` walks from operator to network to subvault. It reduces pending and size at each level. At every tree level it computes the amounts to slash from the *original* `amount`, not from the progressively capped `actualAmount`:

```
for (uint96 curIndex = index; curIndex > 0;) {
    SlotStorage storage slot = slots[curIndex];
    // ▼ uses 'amount' (the original request), not 'actualAmount'
    uint208 pendingSlashed = uint208(Math.min(getPending(curIndex, 0), amount));
    uint128 sizeSlashed = uint128(Math.min(slot.size.latest(), amount - pendingSlashed));
    actualAmount = Math.min(actualAmount, pendingSlashed + sizeSlashed);

    // ▼ state mutations use pendingSlashed/sizeSlashed (derived from 'amount')
    if (pendingSlashed > 0) {
        slot.clearedPendingCursor.push(..., _getPendingCursor(curIndex) + pendingSlashed);
    }
    if (sizeSlashed > 0) {
        slot.size.push(..., slot.size.latest() - sizeSlashed);
    }
    curIndex = curIndex.getParentIndex();
}
```

If `amount` exceeds the operator's allocation, parent nodes have their sizes reduced by more than the actual slash. Concretely, if an operator has 50 allocated but `amount = 100`:

- **Operator level**: slashes min(pending, 100) + min(size, remainder). Reduces by up to 50. `actualAmount` becomes 50.
- **Network level**: slashes min(pending, 100) + min(size, remainder). May reduce by up to 100, *twice* the actual amount being slashed.
- **Subvault level**: same over-reduction.

The returned `actualAmount` is correct (50), but the tree's internal state is corrupted. The sum of children's sizes no longer tracks the parent's size. Prefix sums become invalid. Subsequent allocation queries return wrong results for all operators sharing that network or subvault.

The same issue applies to the shared-subvault guarantee consumption. Both `pendingConsumed` and `sizeConsumed` are computed from `amount` instead of `actualAmount`:

```
uint208 pendingConsumed = uint208(Math.min(_getSharedPendingGuarantee(networkIndex, 0),
    amount));
uint208 sizeConsumed = uint208(Math.min(_getSharedSizeGuarantee(networkIndex), amount -
    pendingConsumed));
```

This inflates the consumed guarantees relative to reality. Subsequent `getAllocated` queries for sibling networks return artificially high values through the chain `_getSharedSizeGuarantee` / `_getSharedPendingGuarantee` to `getAllocated` to `stakeFor` to `slashableStake`.

Code Corrected

The code has been corrected. At each tree level, the reductions now derive from a progressively capped amount rather than the original `amount`.

#019 `skimAdapters` Not Called in `instantWithdraw`

Informational	Version 1	Code Corrected

`deposit`, `withdraw`, and `redeem` all call `skimAdapters()` before computing share amounts. `instantWithdraw` does not.

`syncOwedSlash` also transfers collateral out without calling `deallocateAdapters()` first. Every other transfer-out path (`instantWithdraw`, `claim`, `onSlash`) does.

Code Corrected

The functions are now invoked.

#020 Missing Role Grants in Initialization

Informational	Version 1	Code Corrected

The following roles are not included in `InitParams` and not granted during `_initialize`.

- `VaultV2`: `DEALLOCATE_ADAPTER_ROLE`, `SWAP_ADAPTERS_ROLE`. Without `DEALLOCATE_ADAPTER_ROLE`, collateral can be allocated to adapters but nobody can manually deallocate it.
- `UniversalDelegator`: `REMOVE_SLOT_ROLE`, `SET_WITHDRAWAL_BUFFER_SIZE_ROLE`. Slots can be created but never removed. The withdrawal buffer size set at initialization can never be adjusted.

Code Corrected

The roles are now granted on initialization.

8 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.